

L2 Regularization in Deep Learning (Weight Decay)

Standard Loss Function

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

Minimizes the training loss
(e.g., cross-entropy, MSE).

L2 Regularized Loss Function

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i) + \underbrace{\lambda \sum_l \|W^{(l)}\|_2^2}_{\text{L2 regularization term}}$$

θ : model parameters (weights)

λ : regularization strength ($\lambda > 0$)

$W^{(l)}$: weight matrix of layer l

$\|W^{(l)}\|_2^2$: sum of squared weights in layer l

Effect on Weight Update

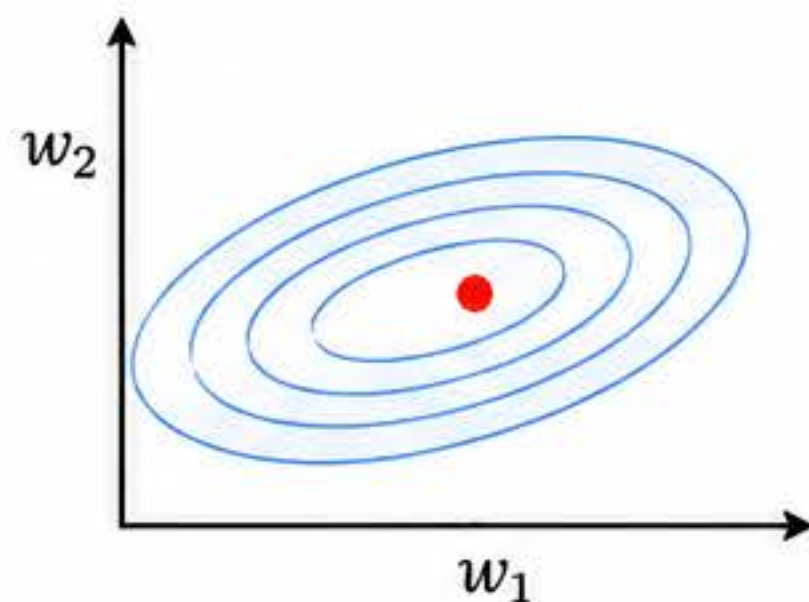
For gradient descent, the update becomes:

$$W \leftarrow W - \eta \left(\frac{\partial L}{\partial W} + 2\lambda W \right)$$

The additional term $2\lambda W$ shrinks the weights towards zero (weight decay).

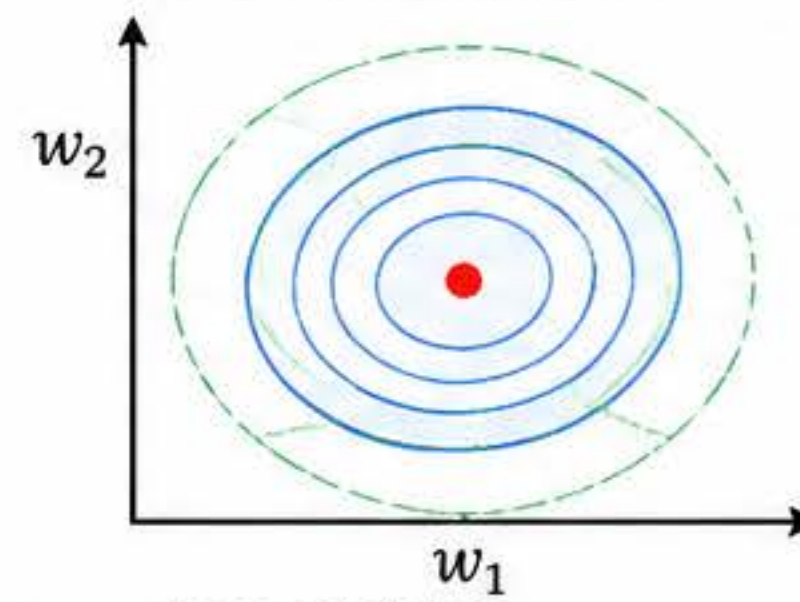
Geometric Intuition

Without Regularization



● Minimum (may have large weights)

With L2 Regularization



— Loss contours
- - L2 constraint ($\|W\|_2^2 = c$)
● Minimum (smaller weights)

Key Effects

- 1 Prevents overfitting**
Discourages very large weights, leading to better generalization.
- 2 Smoother model**
Produces simpler models with smaller weight values.
- 3 Works well in practice**
Widely used in deep neural networks (often called weight decay).
- 4 Controlled by λ**
Larger $\lambda \rightarrow$ stronger regularization (smaller weights).

L2 regularization adds a penalty on the sum of squared weights, encouraging smaller weights and improving generalization in deep learning models.

How Weight Decay (L2 Regularization) Works

1. Model and Loss with L2 Regularization

We minimize the regularized loss:

$$L(\theta) = L_{data}(\theta) + \lambda \|W\|_2^2$$

$L_{data}(\theta)$: training loss (e.g., cross-entropy)

W : weight matrix (parameters)

λ : regularization strength ($\lambda > 0$)

$\|W\|_2^2$: sum of squared weights

2. Gradient of the Regularized Loss

The gradient becomes:

$$\nabla_W L = \nabla_W L_{data} + 2\lambda W$$

The extra term $2\lambda W$ pushes the weights towards zero (proportional to their current value).

3. Weight Update (Gradient Descent)

Update rule with weight decay:

$$W \leftarrow W - \eta (\nabla_W L_{data} + 2\lambda W)$$

$$W \leftarrow \underbrace{(1 - 2\eta\lambda)W}_{\text{Weight decay term (scales weights down)}} - \underbrace{\eta \nabla_W L_{data}}_{\text{Standard gradient step (from data loss)}}$$

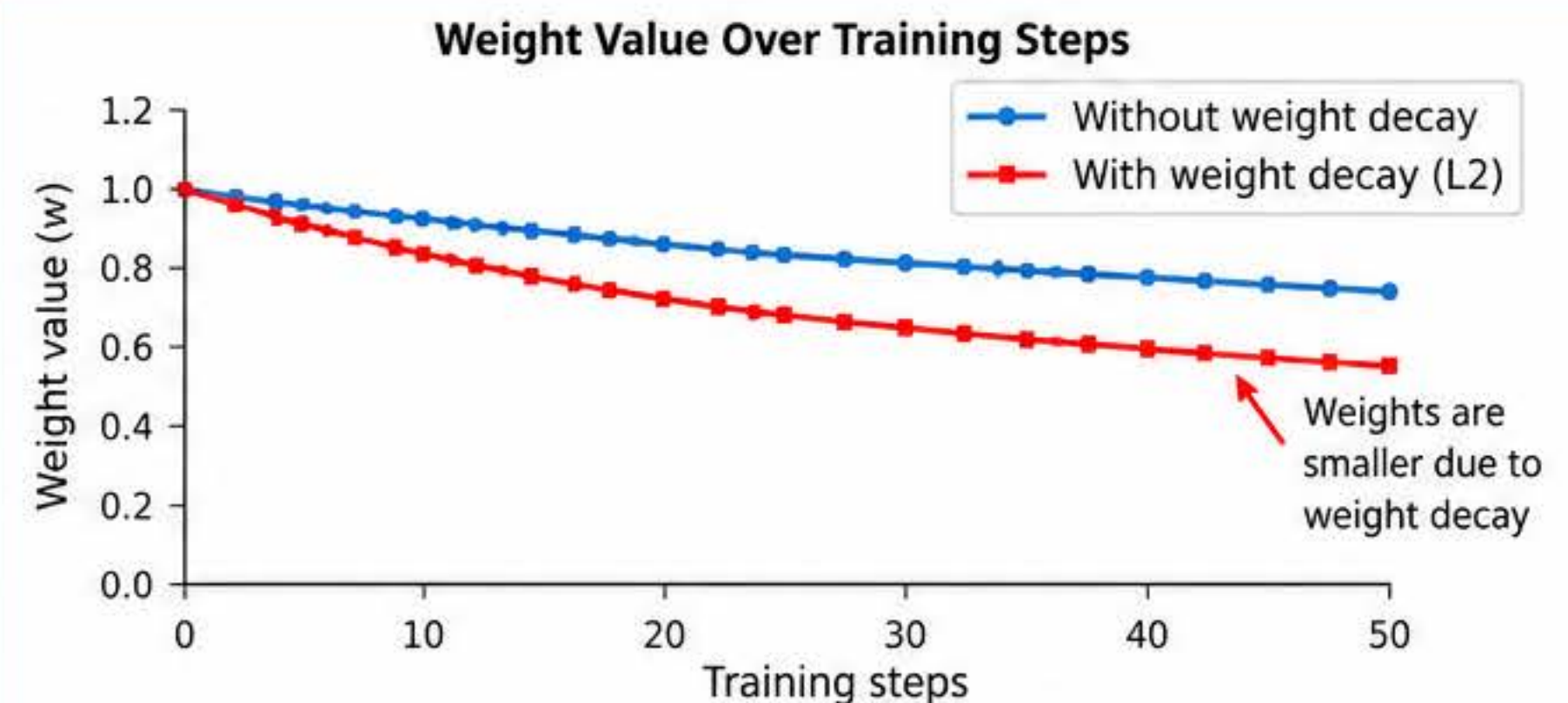
4. Step-by-Step Example (Single Weight)

Assume learning rate $\eta = 0.1$, $\lambda = 0.01$, and gradient from data $\frac{\partial L_{data}}{\partial w} = 0.5$

Step	Without Weight Decay $w \leftarrow w - \eta g$	With Weight Decay $w \leftarrow (1 - 2\eta\lambda)w - \eta g$
Initial w	1.000	1.000
1	0.950	0.948 (= $0.998 \times 1.0 - 0.05$)
2	0.900	0.896
3	0.850	0.845
4	0.800	0.795
5	0.750	0.746

Each update multiplies the weight by $(1 - 2\eta\lambda) < 1$, so the weight gradually shrinks towards zero.

5. Effect on Weights During Training



Key Takeaway

Weight decay adds a term proportional to the current weights in the update, which continuously **shrinks the weights towards zero** during training.